

Roof Internal Library
Project Specification

Project – Roof

Data Foundation Lab
DFLab FHO Pty Ltd – Proprietary Limited Company
(13 676 090 289)

Table of Contents

1. Overview *5*

2. Library Structure *5*

2.1 Current Library Components 5

2.2 Roof.Client 6

2.3 Roof.Common 6

2.4 Roof.NativeAppSDK 7

3. Functional Responsibilities *7*

3.1 Roof.Client Responsibilities 7

3.2 Roof.Common Responsibilities 8

4. System Architecture *8*

4.1 Internal Library Position 8

4.2 Client Library Shape 9

5. API Design Principles *9*

5.1 Consistency 9

5.2 Minimal Surface Area 10

5.3 Secure Defaults 10

5.4 Cross-Platform Compatibility 10

5.5 NativeAOT Compatibility 10

5.6 Integration Simplicity 10

5.7 Event-Driven First, Task-Compatible Later 10

6. Public API Surface *10*

6.1 EntryPoint 10

6.2 Static Events 13

6.3 Static Providers 13

6.4 Default Local TCP Port 13

7. Interface Definitions *13*

7.1 Initialisation 13

7.2 Installation Manager 13

8. Data Model *14*

8.1 Native Request Packet 14

8.2 Native Response Packet 14

8.3 RoofWebSocketPacket 15

8.4 File Metadata DTOs 15

8.5 JSON Contexts 16

9. Local Discovery Flow 16
9.1 Discovery Steps 16
9.2 Presence Constants 17
9.3 NetworkManager Discovery Behaviour 17
10. Local Registration Flow 18
10.1 Validation Requirements 18
10.2 Registration Steps 18
10.3 TCP Encryption Info Handling 19
10.4 Register Request 19
10.5 Registration Completion 19
10.6 Registration Implementation Notes 20
11. Command Execution Flow 21
11.1 Validation Requirements 21
11.2 Local Command Path 21
11.3 PathFinder Command Path 21
11.4 Response Handling 22
11.5 Encryption State Requirement 22
12. RNMS Provider Specification 24
12.1 Supported Protocols 24
12.2 Initialisation Behaviour 24
12.3 Events 25
12.4 Send Overloads 25
12.5 Disposal Behaviour 25
13. Managed TCP Protocol 26
13.1 Frame Format 26
13.2 Data Frame Content 26
13.3 Heartbeat Content 26
13.4 Behaviour 26
13.5 Client Mode 27
13.6 TCP Wrapper 27
14. UDP Protocols 27
14.1 Managed UDP 27
14.2 Unmanaged UDP 28
15. WebSocket Provider Specification 28
15.1 Constants 28

15.2 Behaviour **28**
15.3 Implementation Notes **29**
15.4 PathFinder Provider Ownership **30**
16. Security Handling ***30***
16.1 Client ID Validation **30**
16.2 AES-256 Packet Encryption **31**
16.3 ManagedNetworkEncrypt **31**
16.4 ManagedNetworkDecrypt **31**
16.5 Hashing and Randomness **31**
16.6 Trust Boundary Notes **32**
17. Observability ***32***
17.1 Log Directory **33**
17.2 Log Filename **33**
17.3 Severity Levels **33**
17.4 Behaviour **33**
18. Cross-Platform Support ***34***
18.1 Supported Platform Behaviour **34**
18.2 Current Gap **34**
19. Deployment & Runtime ***34***
19.1 Roof.Client Project Configuration **34**
19.2 Roof.Common Project Configuration **35**
19.3 Local Runtime Lifecycle **36**
19.4 Remote PathFinder Runtime Lifecycle **36**
20. Performance Considerations ***37***
20.1 Lightweight Library Design **37**
20.2 NativeAOT **37**
20.3 Network Discovery Performance **37**
20.4 Transport Performance **37**
21. Testing & Validation ***37***
21.1 Required Test Areas **38**
21.2 API Contract Testing **39**
21.3 Integration Testing **40**
22. Failure Modes & Recovery ***40***
22.1 Client Initialisation **40**
22.2 Presence Discovery **40**

22.3 Registration **40**
22.4 Command Execution **41**
22.5 Transport **41**
23. Risks & Limitations ***42***
Appendices ***45***
Appendix A: Important Constants **45**
Appendix B: Glossary **46**
Appendix C: Acronyms **47**
Appendix D: Local Registration Flow **48**
Appendix E: Command Flow Diagram **49**
Appendix F: RNMS Provider Shape **49**

Version: V1.0.0 (Major, Minor, Patch)
Status: ON-GOING
Owner: Steve Yoo
Last Updated: 09/11/2026

---

1. Overview

The Roof Internal Library layer provides reusable client communication, networking, packet contracts, serialisation, cryptographic helpers and supporting utilities for Roof components.

The current library projects are:

- Roof.Client;
- Roof.Common; and
- Roof.NativeAppSDK.

Roof.Client provides the static client API used to discover local Roof devices, establish local managed TCP connections, register clients, send encrypted commands and deliver asynchronous responses through callbacks. Remote command routing uses a PathFinder provider supplied by the host application.

Roof.Common provides shared request and response models, filesystem metadata models, JSON serialisation contexts, RNMS transport providers, cryptographic helpers, logging and network discovery utilities.

Roof.NativeAppSDK is an existing initial implementation for native application integration through a local RoofDeck bridge. It uses shared models and networking functionality from Roof.Common. It does not independently establish a direct Engine session.

These libraries are implementation foundations rather than standalone end-user applications. Their current limitations and unfinished behaviour must remain distinguished from future SDK and platform requirements.

2. Library Structure

2.1 Current Library Components

Library	Purpose
Roof.Client	Client-facing internal library for Roof discovery, local connection setup, registration, command execution, response handling, and PathFinder command routing.
Roof.Common	Shared foundational library for packet models, RNMS networking, JSON serialisation contexts, encryption, hashing, local discovery, DTOs, and logging.

Library	Purpose
Roof.NativeAppSDK	Existing initial SDK implementation for communicating with a local RoofDeck bridge through managed UDP. Provides login-state checks, filesystem request methods and callback events using shared Roof.Common contracts.

2.2 Roof.Client

Roof.Client builds as a .NET 10 class library with NativeAOT enabled.

It exposes a static entry point through: Roof.Client.EntryPoint

The library currently holds session state in static fields, including:

- active local NetworkMessageProvider;
- active PathFinder NetworkMessageProvider;
- current network encryption key;
- current network encryption IV;
- callback event subscriptions.

Roof.Client is intentionally maintained as a static client surface for the current RoofDeck integration model. RoofDeck loads the client library per application process, meaning the static state is scoped to the active RoofDeck process rather than shared across unrelated application instances.

The current implementation should therefore remain static and process-global. Multiple concurrent Roof sessions within the same process are not currently supported and are outside the current implementation scope.

If future SDK use cases require multi-session support, an instance-based client session model may be introduced at the SDK abstraction layer without changing the current RoofDeck-facing static API.

2.3 Roof.Common

Roof.Common builds as a .NET 10 class library with NativeAOT enabled.

It provides shared functionality for:

- native request and response packets;
- PathFinder WebSocket packet contracts;
- NativeAOT-safe JSON source-generation contexts;
- RNMS transport providers;
- managed TCP framing;
- managed UDP reliability;
- unmanaged UDP transport;
- WebSocket client provider;
- local network discovery;

- file and directory metadata DTOs;
- encryption and hashing helpers;
- logging utilities.

2.4 Roof.NativeAppSDK

Roof.NativeAppSDK is an existing .NET library that references Roof.Common.

Its DefaultProvider communicates with a local RoofDeck bridge using managed UDP. The bridge destination is 127.0.0.1:8672. The SDK binds its own provider to an operating-system-assigned local port.

The SDK includes:

- Initialisation and local bridge connection state;
- RoofDeck process detection;
- logged-in state checks;
- base-directory and directory requests;
- directory creation, update, copy and deletion requests;
- file creation and update requests; and
- filesystem callback events.

In Debug builds, process detection looks for Roof.ClientTestConsole. In IA, Beta and Release builds, it looks for RoofDeck.

The SDK delegates Engine communication to the local bridge. It does not independently perform Engine discovery, registration, credential management or PathFinder connection establishment.

This is an initial implementation. The existence of the project does not establish a complete or stable SDK contract, and declared callback events must not be interpreted as proof that every corresponding operation has a complete public request and response path.

3. Functional Responsibilities

3.1 Roof.Client Responsibilities

Roof.Client is responsible for:

1. Providing the static client entry point used by host applications.
2. Validating required callback subscriptions before client use.
3. Discovering a local Roof device using UDP presence beams.
4. Establishing a managed TCP command connection to a local Roof device.
5. Supporting first-time client registration.
6. Generating valid Roof client identifiers.
7. Maintaining one active process-scoped encryption state used for both outbound command encryption and inbound response decryption.
8. Sending encrypted command packets to Roof Engine.
9. Routing encrypted command packets through PathFinder when required.

10. Receiving, decrypting, deserialising, and forwarding command responses to callback subscribers.
11. Disposing client-side state, network providers, callbacks, and active encryption state.
12. Defining Roof.Client assembly version metadata and providing an internal version helper. The helper reads version information from the process entry assembly and is not exposed as a public Roof.Client API.

3.2 Roof.Common Responsibilities

Roof.Common is responsible for:

1. Defining shared request and response packet contracts.
2. Defining PathFinder WebSocket relay packet contracts.
3. Providing source-generated JSON serialisation contexts for NativeAOT compatibility.
4. Providing the RNMS network provider abstraction.
5. Supporting managed TCP, managed UDP, unmanaged UDP, and WebSocket-based transport modes.
6. Providing managed TCP framing, heartbeat, and disconnection detection.
7. Providing managed UDP ACK/retry reliability.
8. Providing unmanaged UDP fire-and-forget transport.
9. Providing local network discovery by interface enumeration, ping scanning where appropriate, and platform-specific ARP parsing. On macOS, discovery shall avoid active ICMP subnet scanning and instead use usable IPv4 candidates from the ARP table.
10. Providing shared file and directory metadata DTOs.
11. Providing AES-256 encryption and decryption helpers.
12. Providing SHA-256, xxHash64, MD5, nonce generation, and strong password utilities.
13. Providing console and file logging utilities.
14. Providing Boolean RNMS provider initialisation results. Provider initialisation reports success as true and failure as false; it does not currently return structured error details.
15. Defining Roof.Common assembly version metadata and providing an internal version helper. The helper reads version information from the process entry assembly and is not exposed as a public Roof.Common API.
16. Supporting configurable logger output directory paths.

4. System Architecture

4.1 Internal Library Position

The library layer has two current application integration paths.

Applications using Roof.Client communicate with Engine through a local managed TCP provider or a host-supplied PathFinder provider:

Host application or RoofDeck
↓
Roof.Client
↓

Roof.Common packet contracts, encryption and RNMS providers
↓
Local Roof Engine or PathFinder-routed Roof Engine
Applications using Roof.NativeAppSDK communicate through a local RoofDeck bridge:
Native application
↓
Roof.NativeAppSDK and Roof.Common
↓
Managed UDP bridge at 127.0.0.1:8672
↓
RoofDeck or the development ClientTestConsole bridge
↓
Roof.Client and Roof.Common
↓
Local Roof Engine or PathFinder-routed Roof Engine

Roof.NativeAppSDK currently references Roof.Common directly. Its communication with Roof.Client is mediated by the bridge process rather than a direct project reference or an independently managed Engine session.

4.2 Client Library Shape

Roof.Client is currently implemented as a static client surface rather than an instantiable session object.

Current process-global state includes:

- LocalNetworkMessageProvider;
- PathFinderNetworkMessageProvider;
- active process-scoped encryption state;
- callback event subscriptions.

This simplifies the current host integration model, but limits concurrent use. A host application cannot safely manage multiple independent Roof client sessions without shared static state conflicts.

5. API Design Principles

The internal library API should follow these principles:

5.1 Consistency

API behaviour should be consistent across local, remote, and future SDK usage. Packet shapes, response models, encryption expectations, and failure responses should follow predictable conventions.

5.2 Minimal Surface Area
The public surface should expose only the functionality needed by host applications. Internal transport, packet framing, heartbeat logic, and serialisation details should remain hidden unless the caller has a valid integration need.

5.3 Secure Defaults
Encryption, credential handling, transport validation, and sensitive state management should default to the safest available behaviour. Host applications should not need to manually remember avoidable security steps where the library can enforce them.

5.4 Cross-Platform Compatibility
The internal libraries should support Roof's cross-platform runtime model, including macOS, Windows, Linux, and Raspberry Pi deployments where applicable.

5.5 NativeAOT Compatibility
Public packet and DTO models used in runtime serialisation should use source-generated JSON contexts to remain compatible with NativeAOT publishing.

5.6 Integration Simplicity
RoofDeck, PathFinder, Rapps, and future SDK consumers should be able to integrate with the library without duplicating packet encryption, transport framing, or network discovery logic.

5.7 Event-Driven First, Task-Compatible Later
Roof.Client shall remain callback-driven for the current RoofDeck integration model.

Task-returning APIs may be added in future to support safer and cleaner SDK usage. However, task-returning command and registration APIs are dependent on Roof Engine's implementation model, particularly whether Engine responses can be correlated deterministically to individual requests.

Until Engine-level request correlation and response handling are formally supported, callback-based integration remains the authoritative client communication model.

6. Public API Surface
6.1 EntryPoint
EntryPoint exposes the main static client API.

Module	Source Path	Responsibility
EntryPoint	Roof.Client/EntryPoint.cs	Static client API, callback validation, local discovery, local provider setup, command sending, response decryption, and client lifecycle handling.
Installation Manager	Roof.Client/Managers/InstallationManager.cs	Client ID generation and first-time registration handshake.

Module	Source Path	Responsibility
Network Manager	Roof.Common/Networking/NetworkManager.cs	Local IP selection, interface enumeration, subnet scanning, platform-specific ARP parsing, and discovery-candidate filtering.
Native Request Packet	Roof.Common/Networking/RoofNetworkNativeRequestPacket.cs	Client-to-Roof command packet contract and source-generated JSON context.
Native Response Packet	Roof.Common/Networking/RoofNetworkNativeResponsePacket.cs	Roof-to-client response packet contract, duplicate-response suppression, and source-generated JSON context.
Network Message Provider	Roof.Common/Networking/RNMS/NetworkMessageProvider.cs	Transport facade over managed TCP, managed UDP, unmanaged UDP, and WebSocket providers. Initialisation reports success or failure as a Boolean result.
TCP Providers	Roof.Common/Networking/RNMS/TCP/*	Managed TCP transport, length-prefixed framing, heartbeat handling, disconnection monitoring, and wrapper APIs.
UDP Providers	Roof.Common/Networking/RNMS/UDP/*	Managed and unmanaged UDP transports, ACK and retry handling, heartbeat handling, and wrapper APIs.
WebSocket Provider	Roof.Common/Networking/RNMS/WebSocket/*	PathFinder relay packet contract, WebSocket connection handling, heartbeat handling, disconnect detection, malformed-packet tolerance, and cancellation-aware disposal.
Roof Data DTOs	Roof.Common/Networking/RoofData/*	File and directory metadata DTOs and their source-generated JSON contexts.
Security	Roof.Common/Tools/Security.cs	AES encryption and decryption, hashing, nonce generation, client ID validation, and password generation.
Logger	Roof.Common/Tools/Logger.cs	Console and file logging, configurable log-directory selection, public log-path accessors, manual log rotation, and synchronised file writes.
Roof.Client Version Metadata	Roof.Client/AssemblyInfo.cs and Roof.Client/Version.cs	Defines Roof.Client assembly metadata and provides an internal helper that reads the process entry assembly version.
Roof.Common Version Metadata	Roof.Common/AssemblyInfo.cs and Roof.Common/Version.cs	Defines Roof.Common assembly metadata and provides an internal helper that reads the process entry assembly version.

API	Purpose
Initialize()	Performs lightweight client validation only. It currently validates required callback setup and returns a response packet. It does not discover Roof devices, create providers, register clients, or configure encryption state.
SetEncryptionInfo(string key, string iv)	Validates the supplied Base64 key and IV by checking for a decoded 32-byte key and 16-byte IV. Valid values replace the active process-scoped encryption state. The method returns void; invalid values are silently ignored and leave the existing encryption state unchanged.
Dispose()	Removes registration and command callback subscribers, disposes the local network provider, and clears the active encryption key and IV. It does not dispose or clear the host-supplied PathFinder provider. The local provider reference also remains assigned after disposal.
PresenceBeam()	Discovers a local Roof device over UDP.
InitializeNetworkMessageProvider(string destinationAddress)	Creates a local managed TCP provider for destinationAddress:8671 when no local provider is currently assigned. It calls RNMS initialisation but does not inspect its Boolean result, registers local message and disconnection handlers, and returns the provider even when the underlying connection was not established.
SetPathFinderNetworkMessageProvider(NetworkMessageProvider pathfinderNMP)	Supplies a WebSocket-capable provider for remote PathFinder routing.
ExecuteRoofCommand(RoofNetworkNativeRequestPacket requestPacket, bool isThroughPathFinder = false)	Validates, encrypts using the active encryption state, and sends a command packet locally or through PathFinder.
HandleCommandExecutionResultPacket(string packet)	Decrypts a response packet and invokes command callback subscribers.

Initialize() shall remain a lightweight validation method for now. If future versions expand initialisation to include provider setup, discovery, registration preparation, or encryption setup, the method should either be renamed or accompanied by a clearer higher-level initialisation API to avoid ambiguity.

6.2 Static Events
EntryPoint exposes the following callback events: RoofClientRegistrationCallback and RoofCommandExecutionCallback.

Host applications are expected to subscribe to both callback events before initialisation.

6.3 Static Providers
EntryPoint currently maintains: LocalNetworkMessageProvider and PathFinderNetworkMessageProvider.

6.4 Default Local TCP Port
The default local TCP command port is: 8671

7. Interface Definitions
7.1 Initialisation
EntryPoint.Initialize() currently validates callback readiness only.

It does NOT:
- discover a Roof device;
- initialise local network providers;
- initialise PathFinder providers;
- set encryption information;
- register the client.

Required callbacks: RoofClientRegistrationCallback and RoofCommandExecutionCallback
If either callback has no subscribers, initialisation should return a failed response packet.

Implementation Note
Initialize() explicitly validates that both required callback events are non-null and have subscribers.

Required callbacks:
- RoofClientRegistrationCallback;
- RoofCommandExecutionCallback.

If either callback is missing, Initialize() returns a structured failure packet.

7.2 Installation Manager
InstallationManager exposes:

API	Purpose
GenerateUniqueRoofClientID()	Returns a GUID string with hyphens removed.

API	Purpose
RegisterClientToRoof(RoofNetworkNativeRequestPacket requestPacket)	Starts first-time local registration.

Generated client IDs are expected to be 32-character hexadecimal-compatible strings.
Example format: 3f8a2b1c9d0e4f56a7b8c9d0e1f2a3b4

8. Data Model

8.1 Native Request Packet

RoofNetworkNativeRequestPacket represents a client-to-Roof command packet.

Field	Type	Meaning
ClientID	string?	32-character hex client identifier. Required for registration and command execution.
RoofIP	string?	Local Roof IP for registration, or remote Roof identifier/IP for PathFinder routing.
Requests	List<string>?	Command tokens. First item is command prefix; remaining items are command arguments.

Serialisation context: RoofNetworkNativeRequestPacketJSONContext
Indented JSON output is disabled.

8.2 Native Response Packet

RoofNetworkNativeResponsePacket represents a Roof-to-client response packet.

Field	Type	Meaning
Status	bool	Success or failure indicator.
Responses	List<string>	Response tokens or response data.

AddResponse() behaviour:

- initialises Responses when first needed;
- suppresses duplicate response strings.

Responses is declared as a non-nullable List<string> but is initially assigned null. AddResponse() creates the list on first use. Code that reads Responses directly must therefore account for a null value despite the declared non-nullable property type.

Serialisation context: RoofNetworkNativeResponsePacketJSONContext

Indented JSON output is disabled.

8.3 RoofWebSocketPacket

RoofWebSocketPacket is the shared remote relay envelope used by PathFinder routing.

Field	Type	Meaning
SenderIdentifier	string?	Logical sender identifier.
ForwardingToIdentifier	string?	Logical destination identifier used by PathFinder routing.
SenderFingerprint	string?	PathFinder-assigned sender connection fingerprint.
Payload	string?	Caller-defined payload, usually encrypted command data or WS_HEARTBEAT.

Serialisation context: RoofWebSocketPacketJSONContext
Indented JSON output is disabled.

8.4 File Metadata DTOs

RoofDirectoryData

RoofDirectoryData contains:

- HashedPath;
- Name;
- Path;
- Size;
- LastModifiedUTC;
- Files;
- SubDirectories;
- ParentDirectory.

HashedPath is lazily generated from Path using xxHash64 unless explicitly overridden.

RoofFileData

RoofFileData contains:

- HashedPath;
- Name;
- Path;
- Size;
- FileHash;
- LastModifiedUTC;
- ParentDirectory;
- Data;
- ShouldOverwriteData;

- ShouldAppendData.

CompressData() and DecompressData() currently exist as placeholders and do not contain implementation logic.

Several non-nullable DTO properties, including names, paths, hashes, parent-directory values, and file data, do not have constructor-supplied or property-supplied default values.

Newly created DTO instances may therefore contain null values despite their non-nullable declarations.

The HashedPath property is generated lazily from Path. Accessing HashedPath before assigning a valid Path may fail because Path is not initialised by the DTO constructor.

8.5 JSON Contexts

JSON contexts exist for:

- individual file DTOs;
- individual directory DTOs;
- List<string>;
- List<RoofDirectoryData>;
- preserve-reference variants for file and directory DTOs.

File and directory contexts currently use: MaxDepth = 1000

9. Local Discovery Flow

EntryPoint.PresenceBeam() discovers a local Roof device using UDP presence beams.

9.1 Discovery Steps

1. Calls NetworkManager.GetAllLocalNetworkDeviceIPAddresses().
2. Fails if no active IPs are discovered.
3. Fails if the only discovered IPv4 address appears to be a broadcast address ending in .255.
4. Opens a UDP client bound to local port 6968.
5. Enables broadcast on that UDP client.
6. Sends ASCII payload ROOF_PRESENCE_BEAM to every discovered IP at destination port 6969.
7. Waits up to 10 seconds for a response.
8. Uses a 1-second receive timeout per receive attempt.
9. Accepts ROOF_PRESENCE_BEAM_ALIVE as a valid Roof response.
10. Returns the sender IP address in Responses[0].

9.2 Presence Constants

Concern	Value
Client UDP bind port	6968
Roof UDP destination port	6969
Request payload	ROOF_PRESENCE_BEAM
Response payload	ROOF_PRESENCE_BEAM_ALIVE
Overall timeout	10 seconds
Per receive timeout	1 second

9.3 NetworkManager Discovery Behaviour

NetworkManager.GetAllLocalNetworkDeviceIPAddresses() combines:

- local network interface enumeration;
- subnet ping scanning where appropriate;
- ARP table parsing on Windows;
- ARP table parsing on macOS.

macOS discovery behaviour:

- macOS shall avoid active ICMP subnet scanning during local discovery;
- macOS shall return usable IPv4 candidates from the ARP table instead;
- macOS ARP discovery shall use arp -an rather than arp -a;
- arp -an is preferred because it avoids name resolution and returns numeric addresses directly;
- parsed ARP candidates shall be filtered before being returned.

On macOS, addresses obtained from the ARP table are passed through IsUsableIPv4Address() before being returned. The macOS filter excludes:

- non-IPv4 addresses;
- 0.0.0.0;
- loopback addresses in 127.0.0.0/8;
- link-local addresses in 169.254.0.0/16; and
- multicast or reserved addresses whose first octet is 224 or greater.

The current implementation does not apply this filter to successful subnet ping results or addresses parsed from the Windows ARP table. Those results are added directly and deduplicated before being returned.

Interface filtering:

- requires interface status to be OperationalStatus.Up;
- excludes loopback adapters;
- excludes virtual and tunnel adapters.

Virtual adapter keywords include:

- vmware
- virtual
- vethernet
- vbox
- hyper-v

Ping scan values:

Concern	Value
Quick scan timeout	100 ms
Full scan timeout	1000 ms
Maximum concurrent pings per batch	256

GetLocalIPAddress() selection order:

1. first non-virtual IPv4 address on an interface with a gateway;
2. first non-virtual IPv4 address;
3. UDP socket local endpoint selected by connecting to 8.8.8.8:65530;
4. null on failure, despite a non-nullable return type.

10. Local Registration Flow

InstallationManager.RegisterClientToRoof() starts first-time registration.

10.1 Validation Requirements

Registration requires:

- ClientID must not be null or empty;
- RoofIP must not be null or empty;
- ClientID must match: ^[a-fA-F0-9]{32}$
- EntryPoint.LocalNetworkMessageProvider must already be initialised.

10.2 Registration Steps

1. Temporarily removes EntryPoint.ClientNetworkMessageProvider_OnMessageReceived from the local provider.
2. Registers ClientRegistrationNetworkMessageProvider_OnMessageReceived.
3. Stores the request client ID in static registration state.
4. Replaces requestPacket.Requests with: TCP INFO REQUEST
5. Serialises the request through RoofNetworkNativeRequestPacketJSONContext.
6. Sends the unencrypted JSON bytes over local managed TCP.
7. Returns immediate success or failure for packet send only.
8. Final registration success or failure is delivered asynchronously through RoofClientRegistrationCallback.

10.3 TCP Encryption Info Handling

The registration receive handler first determines whether the received message contains valid JSON.

If the message contains JSON, the handler deserialises it as a RoofNetworkNativeResponsePacket and expects Responses[0] to equal: TCP_ENCRYPTION_INFO

The response is then read using the following positions:

Index	Meaning
Responses[1]	Client ID
Responses[2]	Base64 AES-256 key
Responses[3]	Base64 AES IV

The current implementation assumes that Responses is non-null and contains at least four items. A null, empty, or shorter response collection may cause an unhandled exception.

The received key and IV are validated before registration continues. After successful validation, they immediately replace the active process-scoped encryption state.

Registration credentials are not stored separately from normal command-encryption credentials.

10.4 Register Request

After receiving valid TCP encryption information, the client sends an encrypted registration request:

```
1. {
2.   "ClientID": "{clientID}",
3.   "Requests": ["REGISTER REQUEST"]
4. }
```

The packet is:

1. serialised through source-generated JSON;
2. encrypted using the AES-256 helper;
3. sent over the local managed TCP provider.

10.5 Registration Completion

The client decrypts the final registration response using the active process-scoped encryption key and IV.

The expected success response is: RFCLIENT {clientID} REGISTER SUCCESS

On a recognised successful response:

- the active encryption key and IV remain assigned for subsequent command traffic;
- RoofClientRegistrationCallback receives a successful response packet containing the key and IV;
- after callback invocation returns normally, the temporary client ID is cleared;
- the registration receive handler is removed; and
- the normal command-response handler is restored.

The successful callback response contains:

Response index	Meaning
Responses[0]	Active encryption key
Responses[1]	Active encryption IV

InstallationManager does not maintain a separate temporary registration key and IV. Valid credentials received during the TCP information exchange are assigned directly to EntryPoint's active encryption state.

If the final registration response does not match the expected success string, the client clears the active encryption state, invokes the failure callback and returns before clearing the temporary client ID or restoring the normal command-response handler.

Several earlier failure paths also return without complete handler cleanup. There is no general finally-based restoration across every registration outcome. Exceptions from callback subscribers can also interrupt subsequent cleanup.

A new registration attempt does not preserve the previous active credentials. Accepted TCP encryption information replaces them immediately, and a later failure may clear the replacement credentials without restoring the previous values.

Preserving existing credentials during failed registration and guaranteeing cleanup across all outcomes remain future recovery requirements.

10.6 Registration Implementation Notes

The registration receive handler uses async void. Exceptions raised after an awaited operation cannot be observed by the original caller.

Several registration failure paths invoke the registration callback and return without performing complete registration cleanup. These paths may leave the registration receive handler attached, the normal command-response handler detached, or the temporary client ID assigned.

Registration cleanup is therefore guaranteed only when execution reaches the final success cleanup path. Failure recovery may require disposal and recreation of the local client process or provider state.

11. Command Execution Flow

EntryPoint.ExecuteRoofCommand() validates, encrypts, and sends command packets.

11.1 Validation Requirements

Command execution requires:

- active encryption state must exist;
- active encryption key must be valid;
- active encryption IV must be valid;
- request packet must not be null;
- ClientID must be a valid 32-character hex string;
- Requests must exist and contain at least one item;
- Requests[0] must not be null or empty;
- local sends require LocalNetworkMessageProvider;
- PathFinder sends require PathFinderNetworkMessageProvider.

Precondition failures detected by ExecuteRoofCommand() are returned as structured RoofNetworkNativeResponsePacket failures.

The method verifies only that the active encryption key and IV are non-null and non-empty before command processing. It does not revalidate their decoded lengths during each command.

Serialisation, encryption, and some provider send operations are not enclosed in a method-level exception handler. Exceptions raised by those operations may propagate to the caller instead of being converted into a structured response packet.

11.2 Local Command Path

For local command execution:

1. Serialise RoofNetworkNativeRequestPacket.
2. Encrypt serialised JSON through Security.AES256.ManagedNetworkEncrypt() using the active process-scoped encryption state.
3. Send encrypted bytes over local managed TCP.
4. Return immediate send status.
5. Incoming response is later handled by ClientNetworkMessageProvider_OnMessageReceived().

11.3 PathFinder Command Path

For PathFinder command execution:

1. Serialise RoofNetworkNativeRequestPacket.
2. Encrypt serialised JSON through the AES helper using the active process-scoped encryption state.
3. Convert encrypted bytes to UTF-8 string.
4. Wrap the encrypted payload in RoofWebSocketPacket.
5. Serialise RoofWebSocketPacket.
6. Send bytes through PathFinderNetworkMessageProvider.
7. Return immediate send status.

Example packet shape:

```
{
  "SenderIdentifier": "{requestPacket.ClientID}",
  "ForwardingToIdentifier": "{requestPacket.RoofIP}",
  "Payload": "{encryptedRequestBase64}"
}
```

11.4 Response Handling

HandleCommandExecutionResultPacket():

1. Requires active process-scoped encryption state to be set through registration success, session restore, or SetEncryptionInfo().
2. Treats the incoming string as the UTF-8 representation of the Base64 encrypted payload.
3. Decrypts with ManagedNetworkDecrypt() using the active encryption state.
4. Deserialises the decrypted JSON as RoofNetworkNativeResponsePacket.
5. Copies Status and Responses into a new response packet.
6. Invokes RoofCommandExecutionCallback.

11.5 Encryption State Requirement

Command encryption state shall be unified.

Roof.Client shall use one active process-scoped encryption state for both outbound command encryption and inbound response decryption.

Because Roof.Client remains static and process-scoped, the client should behave as having one active Roof session per RoofDeck application process. Encryption key and IV values should therefore be set once after registration or session restore, then reused for all command traffic until they are reset, replaced, or cleared.

Preferred lifecycle:

- SetEncryptionInfo(key, iv)
- ExecuteRoofCommand(requestPacket)
- HandleCommandExecutionResultPacket(packet)
- SetEncryptionInfo(newKey, newIV) when replacing state
- Dispose()

SetEncryptionInfo() returns void and does not report whether the supplied key and IV were accepted. Invalid values are silently ignored, leaving any existing active encryption state unchanged.

The same active key and IV are reused for command encryption and response decryption until the state is replaced or cleared.

The command execution API should avoid requiring key and IV parameters for every outbound command.

The current API shape:

```
1. ExecuteRoofCommand(
2.     RoofNetworkNativeRequestPacket requestPacket,
3.     bool isThroughPathFinder = false
4. )
```

Required behaviour:

Lifecycle Point	Required Behaviour
After successful registration	Store returned key and IV as active encryption state.
After session restore	Restore persisted key and IV through SetEncryptionInfo().
Before command execution	Validate that active encryption state exists.
Outbound command encryption	Use active encryption state.
Inbound response decryption	Use active encryption state.
Session change	Replace active encryption state through SetEncryptionInfo().
Credential rotation	Replace active encryption state through SetEncryptionInfo().
Disposal	Clear active encryption state.
Missing encryption state	Return a structured failure response before serialisation or encryption.
Invalid values supplied to SetEncryptionInfo	Ignore the supplied values and retain the existing active encryption state without returning a status result.

Safety rationale:
A unified encryption state is safer for the current RoofDeck integration model because it:

1. prevents outbound and inbound key/IV mismatch;
2. reduces repeated handling of sensitive values;
3. better matches the static, single-session client design;
4. simplifies command execution for host applications;
5. creates a clearer session lifecycle;
6. allows deliberate key reset or replacement when the active Roof session changes.

Per-command key passing is not the preferred model for the current Roof.Client implementation.

It may only be reconsidered in the future if an SDK-level multi-session or stateless command model requires it.

12. RNMS Provider Specification

NetworkMessageProvider is the shared transport abstraction provided by Roof.Common.

12.1 Supported Protocols

- MANAGED_TCP
- UNMANAGED_UDP
- MANAGED_UDP
- WEBSOCKET_TCP

12.2 Initialisation Behaviour

Protocol	Behaviour
MANAGED_TCP	Creates TCPProvider(destinationAddress, destinationPort, false, isInListenerMode).
UNMANAGED_UDP	Creates UDPProvider(destinationAddress, destinationPort, true, isInListenerMode).
MANAGED_UDP	Creates UDPProvider(destinationAddress, destinationPort, false, isInListenerMode).
WEBSOCKET_TCP	Creates WebSocketProvider, wires events, and connects to new Uri(destinationAddress).

NetworkMessageProvider.Initialize() returns Task<bool>.

NetworkMessageProvider exposes these initialisation overloads:

- Initialize(RNMSNetworkProtocols baseProtocol, bool isInListenerMode, string destinationAddress, int destinationPort = 0)
- Initialize(RNMSNetworkProtocols baseProtocol, bool isInListenerMode, string destinationAddress, int destinationPort, CancellationToken cancellationToken)

The overload without a cancellation token delegates to the second overload using CancellationToken.None.

For WEBSOCKET_TCP, the supplied token is forwarded to WebSocketProvider.Initialize() and linked with the provider-owned cancellation token during connection establishment.

The current TCP and UDP initialisation branches do not use this token to cancel their constructor-based setup. The cancellation overload must therefore not be described as providing equivalent cancellation behavior for every protocol.

During initialisation, the provider records the selected base protocol, creates the corresponding underlying provider, and registers the applicable event handlers.

Initialisation returns true when the selected provider is created successfully. WebSocket initialisation returns the Boolean result produced by WebSocketProvider.Initialize().

Initialisation returns false when the provider configuration is unsupported, a WebSocket connection cannot be established, a destination address or URI is invalid, or an exception occurs during provider creation.

Exceptions raised during provider initialisation are caught and converted to false. The exception and detailed failure reason are not exposed to the caller.
RNMS does not currently return a structured initialisation result.

12.3 Events

NetworkMessageProvider exposes:

- OnClientConnected
- OnClientDisconnected
- OnWebSocketClientConnected
- OnWebSocketClientDisconnected
- OnMessageReceived
- OnWebSocketMessageReceived

12.4 Send Overloads

The provider supports:

- SendMessage(string message)
- SendMessage(string message, IPEndPoint endpoint)
- SendMessage(byte[] messageBuffer)
- SendMessage(byte[] messageBuffer, IPEndPoint endpoint)

12.5 Disposal Behaviour

NetworkMessageProvider.Dispose() disposes the underlying provider selected by the active base protocol.

Disposal behaviour:

- disposes managed TCP providers;
- disposes managed UDP providers;
- disposes unmanaged UDP providers;
- disposes WebSocket providers;
- prevents callback invocation after disposal;
- supports repeated disposal safely and idempotently.
- EntryPoint.Dispose() and the local disconnection handler dispose the current local NetworkMessageProvider. They do not set EntryPoint.LocalNetworkMessageProvider to null.
- As a result, calling InitializeNetworkMessageProvider() after local disposal or disconnection returns the existing disposed provider rather than creating a replacement.

- EntryPoint.Dispose() does not dispose or clear PathFinderNetworkMessageProvider. The host that created and injected the PathFinder provider remains responsible for its disposal.

NetworkMessageProvider does not unregister its wrapper-level event handlers or clear its underlying provider fields during disposal.

At the Roof.Client layer, disposing the local provider does not set EntryPoint.LocalNetworkMessageProvider to null. A later call to InitializeNetworkMessageProvider() therefore returns the previously assigned provider instead of creating a new provider.

The PathFinder provider injected through EntryPoint.SetPathFinderNetworkMessageProvider() remains host-owned and is not disposed by EntryPoint.Dispose().

Managed TCP disposal must cancel or close:

- read loops;
- heartbeat loops;
- disconnection monitor loops;
- active streams;
- active TCP clients.

13. Managed TCP Protocol

Managed TCP is a length-prefixed transport with heartbeat and disconnection monitoring.

13.1 Frame Format

[4-byte big-endian content length][content bytes]

13.2 Data Frame Content

MTCP_DATA:{payload}

13.3 Heartbeat Content

MTCP_HEARTBEAT

13.4 Behaviour

Managed TCP:

- supports listener mode and client mode;
- listener mode binds to IPAddress.Any;
- listener mode accepts multiple clients;
- client mode connects to a single destination endpoint;
- sends heartbeats every 5 seconds;
- monitors disconnection every 150 ms;
- times out after 15 seconds without heartbeat or message activity;

- caps frame content length at 64 MiB;
- emits data frames through MessageReceived;
- ignores heartbeat frames after timestamp update;
- logs unknown frames to console.

13.5 Client Mode

Client mode:

- uses IPAddress.Parse(destinationAddress);
- connects synchronously in the constructor;
- starts the read loop after connection succeeds.

13.6 TCP Wrapper

TCPProvider currently supports managed TCP.

Unmanaged TCP code is present in the API shape but commented out. If created as unmanaged TCP, send methods return false.

14. UDP Protocols

14.1 Managed UDP

Managed UDP provides UDP transport with ACK/retry reliability.

Packet Formats

- MUDP_DATA:{sequence}:{payload}
- MUDP_ACK:{sequence}
- MUDP_HEARTBEAT

Reliability Behaviour

Managed UDP:

- increments sequence numbers using Interlocked.Increment;
- stores sent messages in _pendingMessages;
- immediately ACKs valid data packets;
- removes pending messages on ACK;
- retransmits every 2 seconds;
- retries up to 5 times;
- emits MessageAcknowledged and MessageFailed.

Lifecycle Behaviour

Managed UDP:

- supports listener mode and client mode;
- listener mode binds to the destination port;
- client mode binds to an OS-assigned local port;
- listener mode adds clients lazily when messages arrive;
- client mode disconnects itself if a packet arrives from an unexpected endpoint;
- sends heartbeats every 5 seconds;

- times out after 15 seconds;
- monitors every 100 ms.

14.2 Unmanaged UDP

Unmanaged UDP provides fire-and-forget transport.

Packet Formats

- UMUDP_DATA:{payload}
- UMUDP_HEARTBEAT

Behaviour

Unmanaged UDP:

- does not use ACKs;
- does not retry messages;
- does not use sequence numbers;
- supports listener and client mode;
- adds clients lazily in listener mode;
- disconnects in client mode on unexpected sender;
- sends heartbeats every 5 seconds;
- times out after 15 seconds.

Heartbeat Compatibility

Unmanaged UDP sends and recognises the same heartbeat value: UMUDP_HEARTBEAT

Both paths use the shared HEARTBEAT constant within UnmanagedUDPProvider.

15. WebSocket Provider Specification

WebSocketProvider is the client-side provider used for PathFinder communication.

15.1 Constants

Concern	Value
PathFinder identifier	43627a1de2ca425e9bf877b80dc45725
Heartbeat payload	WS_HEARTBEAT
Heartbeat recurrence	5 seconds
Disconnection timeout	15 seconds

15.2 Behaviour

The WebSocket provider establishes a ClientWebSocket connection and starts receive, heartbeat and disconnection-monitoring tasks after successful connection establishment.

Its current behaviour includes:

- returning false when the connection attempt fails;
- initialising the received-heartbeat timestamp to DateTime.UtcNow when the connection is established;
- sending heartbeat packets approximately every five seconds;
- waiting one second between heartbeat-loop evaluations;
- checking connection and heartbeat state approximately every one second;
- applying a 15-second received-heartbeat timeout, including when the first heartbeat never arrives;
- recognising a heartbeat only when its payload is WS_HEARTBEAT, its sender is the PathFinder identifier and its destination matches the configured client ID;
- treating non-open connection state, close frames and WebSocket failures as disconnection conditions;
- ignoring malformed JSON packets without terminating the receive loop;
- serialising application and heartbeat sends through a shared semaphore; and
- suppressing repeated disconnection events through an atomic flag.

IsConnected reports whether the provider is undisposed, has not raised its disconnection flag and has an open underlying socket.

The initial heartbeat deadline begins when the connection is established. Each recognised incoming heartbeat refreshes that deadline. Ordinary application packets do not refresh the received-heartbeat timestamp.

The host must supply the client identifier used by heartbeat packets through SetClientID(). A missing or incorrect identifier can prevent correct relay identification and heartbeat matching.

15.3 Implementation Notes

WebSocketProvider.Initialize(Uri uri, CancellationToken cancellationToken = default) returns Task<bool>. A failed connection attempt returns false without exposing a structured failure reason.

Connection establishment links the caller’s cancellation token with the provider-owned cancellation token. Background receive, heartbeat and monitoring tasks use the provider-owned token.

The provider uses one send semaphore to prevent overlapping WebSocket sends from application traffic and its heartbeat task.

Malformed JSON packets are skipped. Exceptions raised by application message handlers are caught so that a handler failure does not terminate the receive loop.

The disconnection monitor evaluates connection state and the received-heartbeat deadline at approximately one-second intervals. Heartbeat timeout protection begins at successful connection establishment, including when no initial heartbeat is received.

The close-handshake implementation applies a two-second timeout. Disconnection handling cancels provider work, aborts the socket and suppresses repeated disconnection notifications.

Dispose() cancels provider work, aborts and disposes the socket, and waits up to two seconds for the background tasks. This bounded wait must not be interpreted as an unconditional guarantee that every task has completed before disposal returns.

Connection failure reporting, cancellation, concurrent sends, initial-heartbeat timeout, close-handshake timeout and disposal behavior remain required validation areas.

15.4 PathFinder Provider Ownership

PathFinder provider injection shall remain the current integration model.

The injected PathFinder provider remains owned by the host application. EntryPoint.Dispose() does not dispose the provider or clear PathFinderNetworkMessageProvider.

The host must unsubscribe its own event handlers and dispose the PathFinder provider when the remote session ends.

Host applications are responsible for creating, configuring, and supplying the PathFinder-capable NetworkMessageProvider through: EntryPoint.SetPathFinderNetworkMessageProvider(…)

This keeps Roof.Client transport-agnostic and avoids coupling it too tightly to a specific PathFinder runtime setup.

A higher-level PathFinder setup wrapper should be introduced later to simplify SDK and application usage. That wrapper may handle:

- WebSocket provider creation;
- PathFinder endpoint configuration;
- client identifier setup;
- heartbeat setup;
- connection result reporting;
- reconnection handling;
- structured initialisation failure responses.

16. Security Handling

16.1 Client ID Validation

Client IDs are validated using: ^[a-fA-F0-9]{32}$

InstallationManager.GenerateUniqueRoofClientID() creates a GUID and removes hyphens, producing a compatible 32-character identifier.

16.2 AES-256 Packet Encryption

Security.AES256 provides AES packet encryption and decryption.

AES behaviour:

- generates AES key and IV through Aes.Create();
- returns key and IV as Base64 strings;
- validates AES-256 keys as 32 decoded bytes and 44 Base64 characters;
- validates AES IVs as 16 decoded bytes;
- uses PKCS7 padding;
- uses UTF-8 for plaintext packet JSON;
- outputs encrypted packet bytes as UTF-8 bytes containing Base64 ciphertext.

The implementation creates an Aes instance, assigns the supplied key and IV, and explicitly enables PKCS7 padding. It does not add a message authentication code or authentication tag to the encrypted packet.

The resulting packet format provides encryption but does not independently verify ciphertext integrity or sender authenticity.

Roof.Client reuses the active process-scoped key and IV for command traffic until the encryption state is replaced or cleared.

16.3 ManagedNetworkEncrypt

ManagedNetworkEncrypt(string key, string iv, string data):

1. Base64-decodes key and IV.
2. UTF-8 encodes plaintext data.
3. AES encrypts raw bytes.
4. Base64 encodes ciphertext.
5. UTF-8 encodes Base64 ciphertext for network transport.

16.4 ManagedNetworkDecrypt

ManagedNetworkDecrypt(string key, string iv, byte[] data, int dataLength):

1. UTF-8 decodes received bytes into Base64 ciphertext.
2. Base64-decodes ciphertext.
3. AES decrypts ciphertext.
4. UTF-8 decodes plaintext.

16.5 Hashing and Randomness

Security.SHA256:

- computes SHA-256 for strings and byte arrays;
- returns lowercase hex.

Security.XxHash64:

- computes xxHash64 for byte arrays;

- generates hashed paths from UTF-8 path strings;
- computes stream hashes using a rented 128 KiB buffer;
- preserves stream position by default when seekable;
- validates 16-character hex xxHash64 strings.

ComputeHash(byte[]), GenerateHashedPath(string) and the stream hashing overload create a separate System.IO.Hashing.XxHash64 instance for each call. Hashing operations no longer share a static mutable hasher.

Independent calls therefore do not interfere through shared hashing state. Callers remain responsible for coordinating concurrent access to the same mutable input buffer or stream.

The stream overload hashes from the stream's current position and, by default, restores the original position when the stream supports seeking.

Security.Random:

- GenerateNonce(int length = 32) returns Base64 random bytes;
- GenerateHexNonce(int length = 32) returns lowercase hex random bytes.

Other helpers:

- GenerateMD5Hash(string input);
- GenerateStrongPassword(int length).

16.6 Trust Boundary Notes

First-time registration currently receives AES key and IV in an unencrypted JSON response before the encrypted REGISTER REQUEST. This behaviour shall remain as-is for the current implementation.

Future versions should explore stronger pairing and trust establishment options, including pairing codes, QR-code pairing, signed exchange, trust-on-first-use confirmation, or local physical confirmation. These options are not required for the current implementation but should remain listed as future security improvements.

Encrypted command packets do not include an authentication tag or message authentication code. The current encryption format therefore does not provide explicit tamper detection or proof of sender authenticity.

A future protocol revision should use authenticated encryption, or otherwise add explicit integrity and authenticity protection. The repeated-IV session model should be reviewed as part of that revision.

17. Observability

Roof.Common.Tools.Logger provides console and file logging.

17.1 Log Directory

The default log directory is:

{current working directory}/Logs

Logger.LogFileDirectoryPath exposes the currently selected log directory as a public read-only property.

Logger.SetPreferredLogFileDirectoryPath(string logFileDirectoryPath) replaces the default directory with a host-supplied path.

The preferred directory should be configured before the first file-log operation. The setter does not create the supplied directory immediately; directory creation is performed when file logging first checks the log directory.

17.2 Log Filename

The initial log filename is based on the process start time:

{process start time as yyyy-MM-ddTHHmmss}.log

Logger.LogFilePath exposes the current full log-file path as a public read-only property.

Logger.RotateLogFile() changes the current filename to a timestamp based on the time at which rotation is requested: {rotation time as yyyy-MM-ddTHHmmss}.log

Roof.Common does not schedule rotation automatically and does not implement log compression, age-based eviction, or retention-policy enforcement. These behaviours must be initiated by a consuming application or service.

17.3 Severity Levels

- Neutral
- Warning
- Danger

17.4 Behaviour

Logger:

- writes coloured console output;
- writes debugger output when a debugger is attached;
- appends file output through LogToFile();
- mirrors file logs to console in DEBUG builds;
- logs exception messages, sources, inner exceptions, and stack traces;
- supports a host-configurable preferred log directory;
- exposes the active log directory and current log-file path;
- supports manually requested log-file rotation;
- serialises console operations through a console lock; and
- serialises file writes and filename rotation through a separate file lock.

Automatic rotation timing, compression, and retention are outside the responsibility of Roof.Common.Tools.Logger.

18. Cross-Platform Support

18.1 Supported Platform Behaviour

Current cross-platform behaviours include:

- Windows ARP output parsing;
- macOS ARP output parsing using numeric arp -an output, without active ICMP subnet scanning.
- cross-platform UDP socket fallback for local IP discovery;
- NativeAOT publishing support through source-generated JSON contexts.
- filtering macOS ARP candidates to exclude non-IPv4, zero, loopback, link-local, multicast, and reserved addresses. Equivalent filtering is not currently applied to generic subnet-scan results or Windows ARP results.

18.2 Current Gap

Linux support is required.

Roof hardware is expected to operate on Linux-based environments, including Raspberry Pi deployments. Therefore, local discovery behaviour must support Linux as a first-class runtime target.

NetworkManager.GetAllLocalNetworkDeviceIPAddresses() should be extended to include Linux-compatible local network discovery behaviour, including Linux ARP or neighbour table parsing where appropriate.

Required platform support:

Platform	**Requirement**
Windows	Required
macOS	Required
Linux	Required
Raspberry Pi / ARM64 Linux	Required for Roof hardware runtime

19. Deployment & Runtime

19.1 Roof.Client Project Configuration

- SDK: Microsoft.NET.Sdk;
- target framework: net10.0;
- implicit usings enabled;
- nullable reference types enabled;

- PublishAot = true;
- AllowUnsafeBlocks = true;
- GenerateAssemblyInfo = false;
- configurations: Debug, Release, IA, Beta;
- platform: AnyCPU; and
- project reference: Roof.Common.

Roof.Client assembly metadata is defined manually in AssemblyInfo.cs. AssemblyVersion and AssemblyFileVersion are currently 0.0.0.0.

Informational versions are selected by build configuration:

- Debug: 0.0.0.0-debug;
- IA: 1.0.0.0-ia;
- Beta: 1.0.0.0-beta; and
- Release: 0.0.0.0-release.

The internal Roof.Client.Version.Get() helper reads AssemblyInformationalVersionAttribute from Assembly.GetEntryAssembly(). It therefore reports the process entry assembly's informational version rather than guaranteeing that the value belongs to the Roof.Client assembly.

Roof.Client does not currently expose a public library-version API.

19.2 Roof.Common Project Configuration

- SDK: Microsoft.NET.Sdk;
- target framework: net10.0;
- implicit usings enabled;
- nullable reference types enabled;
- PublishAot = true;
- GenerateAssemblyInfo = false;
- configurations: Debug, Release, IA, Beta;
- platform: AnyCPU; and
- package reference: System.IO.Hashing version 9.0.2.

Roof.Common assembly metadata is defined manually in AssemblyInfo.cs. AssemblyVersion and AssemblyFileVersion are currently 0.0.0.0.

Informational versions are selected by build configuration:

- Debug: 0.0.0.0-debug;
- IA: 1.0.0.0-ia;
- Beta: 1.0.0.0-beta; and
- Release: 0.0.0.0-release.

The internal Roof.Common.Version.Get() helper reads AssemblyInformationalVersionAttribute from Assembly.GetEntryAssembly(). It therefore

reports the process entry assembly's informational version rather than guaranteeing that the value belongs to the Roof.Common assembly.

Roof.Common does not currently expose a public library-version API.

19.3 Local Runtime Lifecycle

1. Host application subscribes to registration and command callbacks.
2. Host calls EntryPoint.Initialize().
3. Host calls EntryPoint.PresenceBeam().
4. Client receives local Roof IP in response packet.
5. Host calls EntryPoint.InitializeNetworkMessageProvider(roofIp).
6. InitializeNetworkMessageProvider() returns the provider object but does not propagate the Boolean RNMS initialisation result. The host cannot infer successful TCP connection establishment from the returned provider object alone.
7. For first-time setup, host generates or sets a client ID.
8. Host calls InstallationManager.RegisterClientToRoof().
9. Client sends unencrypted TCP INFO REQUEST.
10. Roof returns TCP_ENCRYPTION_INFO.
11. Client validates received AES key and IV.
12. Client sends encrypted REGISTER REQUEST.
13. Roof returns encrypted registration result.
14. Client sets returned key and IV as the active process-scoped encryption state.
15. Registration callback receives final key and IV.
16. Host persists key and IV externally if required for session restore.
17. Host calls ExecuteRoofCommand() for future commands.
18. Command responses arrive asynchronously through RoofCommandExecutionCallback.
19. When the local provider disconnects, Roof.Client disposes it but leaves the static provider reference assigned. Automatic reconnection or provider recreation is not currently implemented.

19.4 Remote PathFinder Runtime Lifecycle

1. Host creates and initialises a NetworkMessageProvider using WEBSOCKET_TCP.
2. Host obtains the underlying WebSocketProvider if it needs to call SetClientID().
3. Host supplies the provider through EntryPoint.SetPathFinderNetworkMessageProvider().
4. Host calls ExecuteRoofCommand(..., isThroughPathFinder: true).
5. Client encrypts the native request JSON using the active process-scoped encryption state.
6. Client wraps encrypted payload in RoofWebSocketPacket.
7. PathFinder provider sends the binary WebSocket packet.
8. Incoming WebSocket packets are emitted through OnWebSocketMessageReceived.
9. Host or integration layer passes encrypted response payload into HandleCommandExecutionResultPacket() if using EntryPoint response decryption.

20. Performance Considerations

20.1 Lightweight Library Design

The internal libraries are designed as lightweight class libraries and shared utilities rather than heavyweight runtime services.

20.2 NativeAOT

NativeAOT support reduces runtime reflection dependency and supports smaller, more predictable deployment outputs. This requires disciplined use of source-generated JSON contexts.

20.3 Network Discovery Performance

Network discovery uses:

- interface enumeration;
- ARP parsing;
- subnet ping scanning where appropriate;
- maximum ping concurrency of 256;
- quick scan timeout of 100 ms;
- full scan timeout of 1000 ms.

This allows local discovery to operate quickly, but it also means discovery may generate broad local network traffic.

On macOS, local discovery avoids active ICMP subnet scanning and relies on numeric ARP table parsing through arp -an. This reduces unnecessary active network probing and avoids name-resolution overhead during ARP discovery.

macOS ARP discovery filters unusable IPv4 candidates before returning them. Equivalent filtering is not currently applied to generic subnet ping results or Windows ARP results.

20.4 Transport Performance

Managed TCP uses length-prefixed framing with a 64 MiB frame cap. Managed UDP retries reliable messages up to five times, with a 2-second retransmission interval. WebSocket heartbeat loop includes delay and cancellation behaviour to avoid unnecessary CPU usage and to support clean provider disposal.

The WebSocket disconnect monitor polls every 1000 ms instead of every 100 ms to reduce background CPU churn while still maintaining timely disconnect detection.

21. Testing & Validation

Roof.Client and Roof.Common currently build for .NET 10 in Debug and Release configurations. No dedicated test projects are present for either library.

Nullable reference types are enabled, but compiler warnings remain across packet models, DTOs, providers, events, tasks, and network state. Current warnings include non-nullable

members without initial values, possible null dereferences, possible null returns, null assignments to non-nullable members, and unused exception variables.

Warnings are not currently treated as build errors. A successful build must therefore not be interpreted as confirmation that all nullable contracts are satisfied.

Automated testing and nullable-contract remediation should be completed before treating the internal library layer as stable.

21.1 Required Test Areas

Testing should cover:

1. EntryPoint.Initialize() with null callbacks and populated callbacks.
2. Local presence scan success using a fake UDP responder.
3. Local presence scan failure cases.
4. Registration handshake with simulated managed TCP server.
5. Malformed request packet validation.
6. AES encrypt/decrypt compatibility with Roof Engine.
7. Local command send over managed TCP.
8. PathFinder command wrapping and payload encryption.
9. Response decryption and callback invocation.
10. Managed TCP framing.
11. Managed TCP oversized-frame rejection.
12. Managed TCP heartbeat timeout.
13. Managed UDP ACK/retry/failure behaviour.
14. Unmanaged UDP sends and recognises UMUDP_HEARTBEAT consistently, without the former receive-side spelling mismatch.
15. WebSocket heartbeat and disconnect behaviour.
16. NetworkMessageProvider.Dispose() for each protocol.
17. File and directory DTO serialisation under NativeAOT contexts.
18. Network discovery behaviour on Windows, macOS, and Linux.
19. Static Roof.Client process-scoped behaviour inside a single RoofDeck process.
20. Unified encryption state for outbound and inbound command handling.
21. Encryption state reset or replacement.
22. Corrected ForwardingToIdentifier WebSocket packet field.
23. Failure handling for missing active encryption state.
24. Linux local discovery behaviour.
25. Raspberry Pi / ARM64 Linux runtime validation.
26. Future task-returning API compatibility once Engine-side request correlation is supported
27. RNMS Boolean initialisation results, including confirmation that exceptions and invalid configurations are converted to false.
28. Managed TCP disposal through NetworkMessageProvider.Dispose().
29. WebSocket provider cancellation on disposal.
30. WebSocket heartbeat loop delay behaviour.
31. Library version metadata exposure for Roof.Client.
32. Library version metadata exposure for Roof.Common.
33. Preferred logger directory path setter behaviour.

34. Registration success automatically setting active encryption state.
35. macOS NetworkManager discovery uses ARP table candidates without active ICMP subnet scanning.
36. macOS ARP parsing uses arp -an numeric output.
37. NetworkManager filters unusable IPv4 candidates including loopback, link-local, multicast/reserved, and 0.0.0.0 addresses.
38. WebSocketProvider treats non-open socket state as an immediate disconnect.
39. WebSocketProvider invokes disconnect handling on close frames.
40. WebSocketProvider invokes disconnect handling on WebSocketException.
41. WebSocketProvider continues receiving after malformed JSON packets.
42. WebSocketProvider disconnect monitor uses 1000 ms polling rather than 100 ms polling.
43. EntryPoint.InitializeNetworkMessageProvider() behaviour when RNMS initialisation returns false;
44. local provider reinitialisation after disposal or disconnection;
45. confirmation that EntryPoint.Dispose() leaves the injected PathFinder provider host-owned;
46. registration responses with null, empty, or fewer than four response items;
47. registration handler restoration on every failure path;
48. preservation or clearing of existing encryption state during failed re-registration;
49. propagation of serialisation, encryption, and provider exceptions from ExecuteRoofCommand();
50. WebSocket initialisation starts the 15-second received-heartbeat deadline, and a connection that never receives its first heartbeat enters disconnection handling after that deadline is exceeded and detected by the monitor;
51. Concurrent byte-array and path-based xxHash64 calls produce the same results as independent sequential calls without shared hasher state;
52. Logger path accessors, manual rotation, and concurrent file writes; and
53. nullable-contract tests for packet models, DTOs, provider fields, and events.
54. RNMS forwards the supplied cancellation token to WebSocket connection initialisation.
55. TCP and UDP initialisation are not assumed to support equivalent cancellation behaviour.
56. Application sends and heartbeat sends are serialised through the WebSocket send semaphore.
57. WebSocket close handling is bounded by its two-second timeout.
58. Disposal cancels background work and suppresses subsequent disconnection notifications.

21.2 API Contract Testing

Contract tests should verify that packet formats remain stable across releases.
This should include:

- native request packets;
- native response packets;
- WebSocket relay packets;
- file DTOs;
- directory DTOs;

- list serialisation contexts;
- preserve-reference contexts where required.

21.3 Integration Testing

Integration testing should validate:

- RoofDeck to Roof.Client;
- Roof.Client to Roof.Engine;
- Roof.Client to PathFinder;
- PathFinder-routed command execution;
- local discovery from a real client device;
- Raspberry Pi / Linux runtime behaviour.

22. Failure Modes & Recovery

22.1 Client Initialisation

Failure cases:

- missing callback subscriptions;
- initialisation exceptions.

Callback validation explicitly fails when required callback events are null or have no subscribers.

22.2 Presence Discovery

Failure cases:

- no active network devices found;
- only discovered device appears to be a broadcast address;
- no ROOF_PRESENCE_BEAM_ALIVE response within timeout.

Exceptions while sending presence beams are swallowed and scanning continues.

On macOS, local discovery should avoid active ICMP subnet scanning and use filtered ARP table candidates instead. This reduces network scan activity and avoids relying on ping behaviour that may be restricted, noisy, or inconsistent on macOS networks.

22.3 Registration

Failure cases:

- missing client ID;
- missing Roof IP;
- invalid client ID format;
- local network provider not initialised;
- packet send failure;
- TCP encryption info cannot be deserialised;
- invalid received AES key;
- invalid received AES IV;
- encrypted response cannot be decrypted;

- encrypted response cannot be deserialised;
- final registration response does not match expected success string.
- null or empty response collection;
- TCP encryption information containing fewer than four response items;
- unhandled response-index access;
- registration handler remaining attached after failure; and
- previous active encryption state being replaced or cleared.

Recovery behaviour:

- successful registration clears the temporary client ID, removes the registration handler, and restores the normal command-response handler;
- a final unsuccessful registration response clears the active encryption state but returns before restoring the normal handler;
- several earlier failure paths return without restoring normal handler state;
- an existing active key and IV may be replaced when new TCP encryption information is accepted;
- a later failure may clear the replacement credentials without restoring the previous encryption state; and
- no general finally-based cleanup currently guarantees restoration across all registration outcomes.

22.4 Command Execution

Command execution returns structured failure packets for:

- missing active encryption state;
- a null request packet;
- an invalid client ID;
- a null or empty request list;
- an empty command message;
- a missing local provider;
- a missing PathFinder provider; and
- a provider send operation that completes with a false result.

Serialisation, encryption, and some provider exceptions are not converted into response packets and may propagate to the caller.

Response handling separately catches decryption and JSON deserialisation exceptions and delivers those failures through RoofCommandExecutionCallback.

22.5 Transport

Transport failure behaviours:

- managed TCP, managed UDP, unmanaged UDP, and WebSocket providers emit disconnect events after heartbeat timeout;
- managed UDP retries reliable messages up to five times;
- provider initialisation returns a Boolean success or failure result. Exceptions are caught and converted to false without structured error details.

- NetworkMessageProvider.Dispose() disposes managed TCP providers.
- WebSocket providers treat non-open socket state, close frames, and WebSocketException as disconnect conditions;
- malformed JSON WebSocket packets are skipped without terminating the receive loop;
- WebSocket disconnect monitoring polls every 1000 ms to reduce background churn.

EntryPoint.InitializeNetworkMessageProvider() does not inspect the RNMS Boolean initialisation result and may return a provider whose underlying TCP connection was not established.

WebSocket initialisation sets the received-heartbeat timestamp when the connection is established. A connection that remains open but never receives its first heartbeat is therefore subject to the 15-second heartbeat deadline. The monitor checks approximately once per second and initiates close and disconnection handling when the deadline is exceeded.

Roof.Client disposes its local provider on disconnection but does not clear the static local provider reference or automatically create a replacement.

23. Risks & Limitations

The following risks and limitations are currently identified:

1. Static state makes Roof.Client single-session and difficult to use concurrently.
2. Roof.Client intentionally uses static process-scoped state and does not support multiple simultaneous Roof sessions inside the same process.
3. First-time registration key exchange begins unencrypted and remains accepted for the current implementation.
4. Registration and some provider handlers may still use async void unless separately refactored.
5. Local presence discovery may still send UDP probes broadly to discovered network candidates. On macOS, candidate discovery avoids active ICMP subnet scanning and uses filtered ARP table results instead.
6. Linux ARP or neighbour table parsing still requires implementation if not completed separately.
7. GetLocalIPAddress() can return null despite non-nullable return type if this has not yet been refactored.
8. RoofNetworkNativeResponsePacket.Responses is non-nullable but may be initialised as null if this has not yet been refactored.
9. DTO properties are non-nullable but many have no constructor or default initialisation if this has not yet been refactored.
10. Unmanaged UDP remains internal/experimental.
11. Unmanaged TCP is present in the API shape but is not yet implemented.
12. File data compression and decompression methods remain placeholders.
13. Malformed WebSocket JSON packets are tolerated and skipped to preserve connection stability, but malformed packet frequency should still be logged or monitored where appropriate to detect upstream relay or packet contract issues.

14. Roof.Client does not fully reset its static provider state during disposal. The disposed local provider remains assigned, which prevents the standard initialisation method from creating a replacement provider in the same process.
15. The injected PathFinder provider remains assigned after EntryPoint.Dispose() and must be disposed by its owning host.
16. RNMS initialisation returns only true or false and suppresses exception details.
17. EntryPoint.InitializeNetworkMessageProvider() ignores the RNMS initialisation result and may return a provider without a working underlying connection.
18. Registration response handling assumes that the response collection is non-null and contains the expected number of items.
19. Registration failure can leave the temporary registration handler attached and the normal command-response handler detached.
20. A new registration attempt can replace or clear previously active encryption credentials.
21. Serialisation, encryption, and some provider exceptions may escape ExecuteRoofCommand() instead of being returned as structured failure packets.
22. AES-encrypted packet payloads do not include an authentication tag or message authentication code. Encryption alone therefore does not provide tamper detection or proof of sender authenticity.
23. The active encryption key and IV are reused for command traffic until the encryption state is replaced or cleared.
24. WebSocket heartbeat timeout protection begins at connection establishment. Timeout handling remains dependent on the monitoring task and bounded close/disconnection handling; it is not an exact wall-clock guarantee of closure at fifteen seconds.
25. Unusable-address filtering is applied consistently only to the macOS ARP discovery path. Other discovery paths may return addresses that the macOS path would reject.
26. xxHash64 helpers use a separate hasher per call. Callers must still coordinate concurrent access to the same mutable input buffer or stream.
27. Nullable reference type warnings remain throughout the public and internal networking surface. Runtime null behaviour may differ from the declared non-nullable API contracts.
28. Logger rotation is manually triggered. Roof.Common does not schedule rotation, compress old logs, or enforce a retention period.

Appendices

Appendix A: Important Constants

Constant	Value
Local TCP command port	8671
Presence client bind port	6968
Presence destination port	6969
Presence request	ROOF_PRESENCE_BEAM
Presence response	ROOF_PRESENCE_BEAM_ALIVE
Presence timeout	10 seconds
Managed TCP data prefix	MTCP_DATA:
Managed TCP heartbeat	MTCP_HEARTBEAT
Managed TCP max frame	64 MiB
Managed UDP data prefix	MUDP_DATA:
Managed UDP ACK prefix	MUDP_ACK:
Managed UDP heartbeat	MUDP_HEARTBEAT
Unmanaged UDP data prefix	UMUDP_DATA:
Unmanaged UDP heartbeat, used for sending and receiving	UMUDP_HEARTBEAT
WebSocket heartbeat	WS_HEARTBEAT
PathFinder identifier	43627a1de2ca425e9bf877b80dc45725
Heartbeat recurrence	5 seconds
WebSocket heartbeat-loop polling interval	1 second
WebSocket disconnection-monitor polling interval	1 second
WebSocket close-handshake timeout	2 seconds

Constant	Value
WebSocket disposal background-task wait	Up to 2 seconds
Disconnection timeout	15 seconds
Managed UDP retransmission interval	2 seconds
Managed UDP max retries	5
Ping quick scan timeout	100 ms
Ping full scan timeout	1000 ms
Max concurrent pings	256
Roof.Client library version	Major.Minor.Patch.Build
Roof.Common library version	Major.Minor.Patch.Build
Default log directory	{current working directory}/Logs
Initial log filename	{process start time as yyyy-MM-ddTHHmmss}.log
Rotated log filename	{rotation time as yyyy-MM-ddTHHmmss}.log
Automatic Logger rotation	Not implemented by Roof.Common
Logger retention policy	Not implemented by Roof.Common

Appendix B: Glossary

Term	Definition
Roof	The local device or engine endpoint serving Roof client commands.
Roof Client	An application or library consumer that registers to and controls a Roof.
Client ID	A 32-character hexadecimal identifier generated from a GUID without hyphens.
Native Packet	JSON request or response shape used by Roof command traffic.
RNMS	Shared network message provider abstraction in Roof.Common.
Managed TCP	Length-prefixed TCP transport with heartbeats.

Term	Definition
Managed UDP	UDP transport with sequence numbers, ACKs, retries, and heartbeats.
Unmanaged UDP	Fire-and-forget UDP transport with heartbeat and disconnect tracking.
PathFinder	Remote WebSocket relay for non-local Roof communication.
RoofWebSocketPacket	Relay envelope used for PathFinder routing. Uses ForwardingToIdentifier as the destination routing field.
Presence Beam	UDP discovery request used to locate a local Roof.
Hashed Path	xxHash64 identifier derived from a file-system path.
NativeAOT	.NET ahead-of-time publishing model requiring explicit serialisation contexts for reliable runtime behaviour.
Active Encryption State	The process-scoped AES key and IV currently used by Roof.Client for outbound command encryption and inbound response decryption.
Boolean Initialisation Result	A true or false value returned by RNMS provider initialisation. The current implementation does not return a structured failure code, message, or captured exception.
Library Version Metadata	Assembly version attributes defined by Roof.Client and Roof.Common. The accompanying internal version helpers read the process entry assembly and are not public library-version APIs.
Preferred Logger Directory	A host-configurable directory path used by Roof.Common.Tools.Logger for file log output.
Authenticated Encryption	An encryption construction that provides confidentiality together with integrity and authenticity verification. The current Roof AES packet format does not provide an authentication tag.
Logger Rotation	A manually requested change to the current timestamp-based log filename through Logger.RotateLogFile(). Roof.Common does not schedule rotation automatically.

Appendix C: Acronyms

Acronym	Meaning
AES	Advanced Encryption Standard

Acronym	Meaning
API	Application Programming Interface
ARP	Address Resolution Protocol
DTO	Data Transfer Object
IV	Initialisation Vector
JSON	JavaScript Object Notation
RNMS	Roof Network Message System / Network Message Provider abstraction
SDK	Software Development Kit
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
URI	Uniform Resource Identifier
WebSocket	Full-duplex communication protocol over TCP

Appendix D: Local Registration Flow

```
sequenceDiagram
    participant App as Host App
    participant Client as Roof.Client
    participant TCP as Local Managed TCP
    participant Roof as Roof.Engine

    App->>Client: Subscribe callbacks
    App->>Client: Initialize()
    App->>Client: PresenceBeam()
    Client->>Roof: UDP ROOF_PRESENCE_BEAM
    Roof-->>Client: UDP ROOF_PRESENCE_BEAM_ALIVE
    App->>Client: InitializeNetworkMessageProvider(roofIp)
    App->>Client: RegisterClientToRoof(clientId, roofIp)
    Client->>TCP: TCP INFO REQUEST
    TCP->>Roof: Unencrypted JSON request
    Roof-->>TCP: TCP_ENCRYPTION_INFO, key, IV
    TCP-->>Client: Unencrypted JSON response
    Client->>TCP: Encrypted REGISTER REQUEST
    TCP->>Roof: AES encrypted request
    Roof-->>TCP: AES encrypted register result
    TCP-->>Client: Result packet
    Client-->>App: RoofClientRegistrationCallback(key, IV)
```

Appendix E: Command Flow Diagram

```
flowchart TD
    A["Host calls ExecuteRoofCommand"] --> B{"PathFinder?"}
    B -- "No" --> C["Serialise native request"]
    C --> D["Encrypt with active encryption state"]
    D --> E["Send over managed TCP"]
    B -- "Yes" --> F["Serialise native request"]
    F --> G["Encrypt with active encryption state"]
    G --> H["Wrap in RoofWebSocketPacket using ForwardingToIdentifier"]
    H --> I["Send over WebSocket provider"]
    E --> J["Roof processes command"]
    I --> J
    J --> K["Encrypted response payload"]
    K --> L["HandleCommandExecutionResultPacket"]
    L --> M["Decrypt with active encryption state and deserialise response"]
    M --> N["Invoke RoofCommandExecutionCallback"]
```

Appendix F: RNMS Provider Shape

```
flowchart LR
    NMP["NetworkMessageProvider"] --> TCP["TCPProvider"]
    NMP --> UDP["UDPProvider"]
    NMP --> WS["WebSocketProvider"]
    TCP --> MTCP["ManagedTCPProvider"]
    TCP -. "commented / false" .-> UTCP["Unmanaged TCP"]
    UDP --> MUDP["ManagedUDPProvider"]
    UDP --> UUDP["UnmanagedUDPProvider"]
    WS --> CWS["ClientWebSocket"]
```